

Machine Learning Case Studies: Missing Data, Imbalanced Data, and Drift Detection

Ashish Mishra

2025

Abstract

This article presents three machine learning case studies demonstrating practical approaches to handling missing data, imbalanced datasets, and both data and concept drift in real-time systems. These scenarios illustrate applied problem-solving techniques relevant to production-level machine learning pipelines.

1 Scenario 1: Handling Missing Data

In a credit risk prediction dataset containing 100,000 applicants, approximately 10% of the records had missing values in critical fields such as income and employment length. Because these missing values correlated with applicant age, the pattern was not completely random and was best categorized as Missing At Random (MAR).

To address this problem effectively, the following strategy was used:

- **Diagnose Missingness:** Analyze missingness correlations, visual patterns, and demographic associations.
- **Model-Based Imputation:** Use Random Forest or LightGBM imputers for continuous variables such as income.
- **Categorical Imputation:** Apply grouped mode imputation or ordinal models for employment length.
- **Add Missingness Indicators:** Create binary flags to preserve information inherent in missing patterns.
- **Cross-Validated Pipeline:** Perform cross-validation on the full pipeline to avoid leakage.

- **Fairness and Sensitivity Analysis:** Evaluate subgroup impacts and distributional shifts caused by imputation.

This approach balances accuracy, fairness, and interpretability while minimizing data loss.

2 Scenario 2: Handling Imbalanced Data

A fraud detection system with 1,000,000 transactions and only 0.5% labeled as fraud required a strategy suitable for extreme class imbalance. Because false negatives are costly in fraud analysis, the pipeline prioritized recall while maintaining operational precision.

Key steps included:

- **Stratified Splitting:** Maintain class ratios in training and validation.
- **Class Weighting:** Apply model-based weighting such as LightGBM's `scale_pos_weight`.
- **Feature Engineering:** Create time-window features, event rate features, and categorical encodings.
- **Model Selection:** Favor gradient boosting models (LightGBM, XGBoost) for tabular imbalance.
- **Threshold Optimization:** Tune decision thresholds based on recall, F2-score, and precision–recall AUC.
- **Cost-Based Evaluation:** Assess the system using a cost matrix that reflects the high penalty of missed fraud.

This methodology ensures rigorous evaluation and effective mitigation of imbalance without relying excessively on synthetic oversampling.

3 Scenario 3: Detecting and Mitigating Data Drift and Concept Drift

A real-time recommendation system requires constant adaptation due to shifting user behavior, seasonality, and evolving item popularity. Both *data drift* (changes in feature distributions) and *concept drift* (changes in relationships between features and outcomes) must be detected and managed.

Effective measures included:

3.1 Detecting Data Drift

- Distributional tests such as KS, Chi-Square, and PSI.
- Monitoring embedding drift through cosine distances.
- Traffic anomaly detection in event streams.

3.2 Detecting Concept Drift

- Monitoring declines in CTR, NDCG, and calibration.
- Maintaining shadow models for performance comparison.
- Applying drift detectors such as DDM and ADWIN.

3.3 Mitigating Drift

- Frequent retraining and warm-start modeling.
- Refreshing user and item embeddings.
- Using contextual bandits for fast adaptation.
- Implementing seasonal or time-aware feature engineering.

3.4 Deployment and Monitoring

- Shadow deployment followed by canary rollout.
- Continuous monitoring of fairness, stability, and performance.
- Evaluation using both offline metrics (NDCG, Recall@K) and online metrics (CTR, retention).

This comprehensive approach ensures long-term system robustness and fairness.

4 Conclusion

These three scenarios illustrate essential competencies in modern machine learning practice: handling missing data responsibly, building systems for extreme class imbalance, and designing robust drift-aware pipelines. Together, they demonstrate the importance of fairness, transparency, and continuous monitoring in real-world AI systems.